MVC (Ejemplo práctico)
Introducción y conceptos básicos

Explicación del patrón MVC:

MVC (Model-View-Controller) es un patrón de diseño de software que separa la lógica de la aplicación en tres componentes principales:

- **Model (Modelo):** Representa los datos y la lógica de negocio. Se encarga de gestionar y manipular la información.
- **View (Vista)**: Es la interfaz que el usuario ve e interactúa. Presenta los datos del modelo y se encarga de la parte visual.
- **Controller (Controlador):** Actúa como un intermediario entre el modelo y la vista. Recibe las entradas del usuario (por ejemplo, a través de formularios) y decide qué debe hacer el sistema.

Explicación del árbol de directorios:

Directorios y Archivos:

1. **`/.htaccess`**: Archivo de configuración para el servidor Apache. Define reglas de reescritura (URL rewriting) para que las rutas del sitio no necesiten una extensión ".php" o para redirigir tráfico de manera adecuada. Cada carpeta clave del proyecto tiene uno para controlar cómo se manejan las rutas.
2. **`/app`**: Aquí reside toda la lógica de la aplicación.
   - **`/controllers`**: Contiene los controladores de la aplicación, que son los que manejan la lógica de la aplicación. Ejemplo: `HomeController.php`, `SiteController.php`.
   - **`/core`**: Esta carpeta contiene las clases centrales o base del framework que estás desarrollando. Estas son clases reutilizables para autoloading, controladores base, conexión a la base de datos, etc.
     - `App.php`: Clase que maneja el enrutamiento de la aplicación.
     - `Autoloader.php`: Es responsable de cargar automáticamente las clases necesarias cuando se instancian.
     - `Controller.php`: La clase base de la que heredan los controladores.
     - `DataBase.php`: Clase que maneja la conexión a la base de datos.
     - `Model.php`: La clase base para los modelos.
     - `Response.php`: Clase encargada de manejar las respuestas HTTP (status codes, headers, etc.).
   - **`/models`**: Aquí irán los modelos de la aplicación, responsables de manejar la lógica de los datos.
   - **`/views`**: Contiene las vistas o interfaces visuales que el usuario verá. Se divide en carpetas para organizar las vistas por áreas:
     - `/home`: Contiene vistas relacionadas con la página de inicio, como `inicio.php` y una página de error `404.php`.
     - `/inc`: Contiene partes comunes de las vistas, como el archivo `head.php`, que podría incluir las meta etiquetas, títulos y enlaces a hojas de estilo.
3. **`/public`**: Es el directorio accesible públicamente.
   - `index.php`: Es el punto de entrada de la aplicación. Es probable que este archivo redirija todas las solicitudes a la aplicación.
   - **`/css`**: Carpeta de archivos CSS que contiene la definición de estilos como `grid.css`, `main.css`, y `reset.css`.
   - **`/img`**: Carpeta para imágenes utilizadas en el proyecto.
   - **`/js`**: Contiene scripts JavaScript, como `jquery.js` y `main.js`.

Explicación detallada de los archivos clave:

1. `App.php` (Clase para manejar rutas y controladores)

Este archivo contiene la clase `App`, que es esencial para gestionar el flujo de la aplicación. Aquí se maneja el enrutamiento y la invocación de los controladores y métodos correspondientes. A continuación, se describe su funcionamiento:

- **Propiedades**:
  - `protected $controller`: Define el controlador por defecto, en este caso, `HomeController`.
  - `protected $method`: Método por defecto, `actionIndex`.
  - `protected $params`: Parámetros adicionales de la URL.
- **`__construct()`**: Este es el constructor de la clase, que es el responsable de procesar la URL, cargar el controlador correspondiente, y ejecutar el método adecuado basado en los parámetros de la URL.
  - **parseUrl()**: Se encarga de extraer y sanitizar la URL.
  - **controllerExists()**: Verifica si el controlador solicitado existe en el sistema.
  - **getControllerClass()**: Convierte el nombre del controlador en su correspondiente formato de espacio de nombres.
  - **getMethodFromUrl()**: Extrae el método desde la URL, buscando el segundo segmento (después del nombre del controlador).
  - Si no se encuentra el controlador o el método, se invoca el método `action404`.

2. `HomeController.php` (Controlador principal)

Este archivo contiene la clase `HomeController`, que hereda de la clase base `Controller`. Aquí se definen las acciones relacionadas con la página de inicio.

- **`__construct()`**: Constructor vacío en este caso, pero preparado para ser utilizado en futuras expansiones.
- **`actionIndex()`**: Método que actúa como la acción por defecto y simplemente imprime un mensaje: "hola desde index de home".
- **`actionInicio()`**: Método de ejemplo que renderiza una vista específica `inicio.php`, utilizando la clase `Response::render()` para generar la página.
- **`action404()`**: Método que maneja el error 404 (página no encontrada), renderizando una vista de error.

Este controlador sigue el patrón típico de MVC, donde las acciones dentro de los controladores se encargan de coordinar entre el modelo y la vista.

3. `index.php` (Punto de entrada)

Este archivo actúa como el punto de entrada de la aplicación. Es el que recibe todas las solicitudes desde el navegador y decide qué hacer con ellas.

- **Configuración inicial**:
  - Activa la sesión con `session_start()`.
  - Configura la visualización de errores para desarrollo con `error_reporting(E_ALL)` y `ini_set('display_errors', 1)`.
- **Definiciones de rutas**: Define constantes que representan las rutas principales del proyecto, como `CORE_PATH`, `APP_PATH`, y `ROOT_PATH`.
- **Carga del Autoloader**: Incluye el archivo `Autoloader.php` desde el directorio de `core`. El autoloader es esencial para cargar dinámicamente las clases a medida que se necesiten, evitando tener que incluir manualmente cada archivo.
- **Inicialización de la aplicación**: Finalmente, se instancia la clase `App()`, lo que inicia el enrutamiento y carga el controlador y método adecuados.

El **autoloading** en PHP es una técnica que permite cargar las clases automáticamente cuando son necesarias, sin tener que incluir manualmente cada archivo. Esto es crucial en aplicaciones grandes, ya que simplifica la gestión de dependencias y mejora el rendimiento del desarrollo.

archivo `Autoloader.php`

El **autoloader** es una pieza fundamental en cualquier aplicación que siga el patrón MVC, ya que automatiza la carga de clases cuando estas son necesarias. El archivo `Autoloader.php` en tu proyecto sigue una estructura clara y eficiente para hacer esto. ¿cómo funciona?

1. Clase `Autoloader`

Esta clase se encarga de registrar y manejar el autoloading de las clases. El autoloading es una técnica que permite cargar automáticamente los archivos de clase cuando se instancian, en lugar de tener que incluirlos manualmente cada vez.

- **Constructor `__construct()`**: El constructor llama al método `loadAppClasses()` para registrar la función que se encargará de cargar automáticamente las clases. Esto asegura que cada vez que se instancia una clase no previamente cargada, PHP sabrá cómo localizar y cargar el archivo correspondiente.

2. Método `loadAppClasses()`

Este método utiliza `spl_autoload_register`, una función interna de PHP que permite registrar una función de autoload personalizada. Cada vez que se intente usar una clase, PHP llamará a la función registrada para buscar y cargar la clase correspondiente.

- **Registro de clases**: Se define un array llamado `$clasesParaCargaAutomatica` que contiene los nombres de las clases clave que deben ser cargadas automáticamente (`App`, `Controller`, `Model`, `Response`, `DataBase`). El código busca coincidencias entre el nombre de la clase que se está solicitando y las clases listadas en este array.
- **Proceso de autoload**: Si se encuentra una coincidencia entre el nombre de la clase solicitada y las clases en `$clasesParaCargaAutomatica`, se carga el archivo correspondiente utilizando `require_once`. El nombre de la clase se convierte en una ruta válida utilizando `str_replace('\\', '/', $nombreClase)`, lo que permite localizar el archivo dentro de las carpetas del proyecto.
- **Manejo de errores**: Si la clase solicitada no está en la lista de `$clasesParaCargaAutomatica`, se lanza una excepción con un mensaje claro: `No se pudo cargar la clase: $nombreClase`. Esto ayuda a depurar errores cuando no se puede encontrar la clase solicitada.

3. Instancia del Autoloader

Finalmente, fuera de la clase `Autoloader`, se instancia la clase llamando a `new Autoloader();`. Esto asegura que el autoloader esté activo desde el inicio de la aplicación, permitiendo cargar automáticamente todas las clases que se soliciten sin necesidad de incluir manualmente cada archivo.

Resumen:

- El **Autoloader** es responsable de cargar automáticamente las clases de la aplicación cuando se necesitan.
- Utiliza el registro de la función `spl_autoload_register()` para buscar y cargar las clases desde sus archivos correspondientes.
- Las clases que se deben cargar automáticamente se definen en un array (`$clasesParaCargaAutomatica`), y si la clase solicitada no está en ese array, se genera un error.

Este mecanismo es esencial para simplificar el desarrollo, evitando que el código se llene de declaraciones `require` o `include`.

Explicación del Modelo en tu aplicación MVC

1. `Model.php` (Clase base para los modelos)

Este archivo contiene la clase `Model`, que es una clase base genérica utilizada por otros modelos, como el `UserModel`. Su objetivo principal es proporcionar métodos comunes para interactuar con la base de datos, lo que simplifica el desarrollo y evita la duplicación de código.

Funciones principales:

- `protected $table`: Esta propiedad define la tabla de la base de datos con la que el modelo interactuará.
- `protected $primaryKey = "id"`: Define la clave primaria por defecto, que es comúnmente `id`, aunque los modelos específicos pueden sobrescribirla si es necesario.
- `findId($id)`:
  - Este método busca un registro en la base de datos basado en el ID.
  - Genera una consulta SQL para seleccionar un registro de la tabla correspondiente a partir del valor de su clave primaria.
  - Los valores del registro recuperado se asignan a las propiedades del modelo.
- `getAll()`:
  - Recupera todos los registros de la tabla asociada al modelo.
  - Ejecuta una consulta SQL simple `SELECT *` y devuelve los resultados.
- `getColumnsNames($table)`:
  - Devuelve los nombres de las columnas de una tabla específica. Esto es útil para obtener metadatos de la base de datos, como generar formularios o realizar validaciones.

Esta clase base sigue el patrón de **Active Record**, donde el modelo no solo representa una tabla de la base de datos, sino que también incluye métodos para interactuar con ella directamente.

2. `UserModel.php` (Modelo específico de usuarios)

Este archivo define la clase `UserModel`, que extiende de la clase base `Model`. Está especializado en la tabla `usuarios` y contiene métodos específicos relacionados con la funcionalidad de los usuarios.

Funciones y propiedades:

- `protected $table = "usuarios"`: Define que este modelo está vinculado a la tabla `usuarios`.
- `protected $primaryKey = "id"` y `protected $secundaryKey = "email"`: El modelo usa la clave primaria `id`, pero también tiene una clave secundaria `email`, lo que sugiere que se puede buscar un usuario tanto por su ID como por su correo electrónico.
- `findEmail($email)`:
  - Este método busca un usuario basado en su dirección de correo electrónico.
  - Realiza una consulta SQL `SELECT * FROM usuarios WHERE email = :email` para buscar el registro.
  - Si encuentra un registro, asigna los valores a las propiedades del modelo.
- `changePassword($newPass, $token)`:
  - Este método actualiza la contraseña de un usuario en la tabla `usuarios` utilizando un token de seguridad.
  - Realiza una transacción con la base de datos para asegurar que el cambio se realice de forma correcta y segura.
  - Si la transacción falla, captura la excepción y devuelve el mensaje de error.

Resumen:

- **Modelo**: Es la parte de la aplicación que gestiona los datos y la lógica de negocio. En este ejemplo de proyecto, los modelos heredan de una clase base (`Model.php`) que proporciona métodos comunes como `findId` y `getAll`, simplificando el acceso a la base de datos.
- **UserModel**: Es un modelo específico para manejar los datos de usuarios. Incluye métodos personalizados como `findEmail` y `changePassword` que interactúan con la base de datos y aplican la lógica específica para la gestión de usuarios.

Explicación de cómo interactúan el Controlador y el Modelo: `UserController.php` y `UserModel.php`

El archivo **`UserController.php`** actúa como el intermediario entre las solicitudes del usuario y la lógica de negocio gestionada por el **modelo** (`UserModel.php`). Este controlador gestiona varias acciones relacionadas con la obtención y verificación de datos de usuarios.

1. Estructura del Controlador:

El `UserController` hereda de la clase `Controller`, lo que sugiere que utiliza funcionalidades comunes que existen en la clase base `Controller`. Dentro del controlador, se implementan varias funciones para interactuar con los datos de usuarios a través de `UserModel`.

Métodos Clave:

- **`actionIndex($var = null)`**:
  - Esta es la acción por defecto que se ejecuta cuando no se especifica un método en particular. En este caso, simplemente llama a `action404()` para manejar una URL no válida.
- **`GetUser($emailOrId)`**:
  - Este método es responsable de obtener los datos de un usuario, ya sea por **email** o **ID**.
  - Usa el método `findEmail()` del modelo si el parámetro es un correo electrónico válido (validado por `filter_var`), y si no, usa `findId()` para buscar por ID.
  - Devuelve los datos del usuario correspondientes.
- **`GetUserbytoken($token)`**:
  - Busca un usuario por su **token** (probablemente un token de seguridad para identificar al usuario de manera única) llamando a un método `GetUserbytoken()` en el modelo `UserModel`.
- **`checkActivo($userEmail)`**:
  - Este método verifica si un usuario con un email específico está activo.
  - Utiliza `UserModel::findEmail()` para obtener los datos del usuario y revisa si el campo `activo` está configurado como "sí".
  - Si el usuario está activo, devuelve `true`, de lo contrario, devuelve un mensaje de error personalizado.

2. Relación entre Controlador y Modelo:

El controlador utiliza métodos del modelo `UserModel` para realizar consultas a la base de datos y manejar la lógica asociada a los usuarios. Este patrón es común en el **MVC**: el controlador **no** se encarga de acceder directamente a la base de datos, sino que delega esta responsabilidad al modelo, manteniendo una separación clara de responsabilidades.

Flujo típico:

1. El **controlador** recibe una solicitud (por ejemplo, buscar un usuario por email o ID).
2. El controlador llama al **modelo** (`UserModel`) para obtener los datos del usuario correspondiente.
3. Una vez que el modelo recupera los datos, el controlador puede:
   - Retornar esos datos (o pasarlos a una vista para mostrarlos al usuario).
   - Realizar validaciones adicionales (como verificar si el usuario está activo).

Ejemplo Práctico:

Si un usuario quiere recuperar su perfil utilizando su email, el flujo sería el siguiente:

1. El controlador `UserController` llama a `GetUser($emailOrId)`.
2. Dentro de `GetUser()`, se valida si el parámetro es un email o un ID.
   - Si es un email válido, se llama a `UserModel::findEmail($email)`, que consulta la base de datos y devuelve los datos del usuario.
   - Si es un ID, se llama a `UserModel::findId($id)` para realizar la consulta.
3. Los datos del usuario se devuelven al controlador, que puede realizar otras acciones o enviarlos a una vista.

Resumen:

- El **controlador** es el intermediario que coordina la lógica entre el usuario (interfaz/vista) y el modelo (datos).
- El **modelo** maneja toda la lógica relacionada con los datos, como acceder a la base de datos y devolver la información solicitada.
- En este caso, el `UserController` usa el `UserModel` para obtener los datos de los usuarios, verificar si están activos, o recuperar usuarios a través de su email o token.

Esto demuestra una implementación clara del patrón **MVC**, donde cada componente tiene una responsabilidad bien definida.

Las Vistas en un proyecto MVC

Utilizando los archivos `inicio.php` y `head.php`, vamos a analizar cómo interactúan las vistas con el controlador y cómo se renderizan los datos para ser mostrados al usuario.

1. Archivo `inicio.php`: Página de inicio

Este archivo es una de las vistas asociadas con la acción `actionInicio()` del `HomeController` o cualquier otro controlador que lo invoque. Aquí es donde se renderiza el contenido que el usuario ve al acceder a la página de inicio.

Contenido de `inicio.php`:

```
<!DOCTYPE html>
<html lang="es">
<head>
    <?=$head?>
    <title><?=$title?></title>
</head>
<body>
    <div class="row">
        <div class="col_12">
            <h1>Hola desde Inicio</h1>
        </div>
    </div>
</body>
</html>
```

Análisis:

- `<?=$head?>`: Esta línea incluye el contenido del archivo `head.php`. Utiliza PHP para incrustar código de la vista que contiene los elementos `<meta>`, hojas de estilo, y scripts necesarios para la página. Es una forma de reutilizar código en varias vistas (en este caso, la cabecera del HTML).
- `<?=$title?>`: Se utiliza para mostrar el título de la página. El controlador es el encargado de pasar el valor para esta variable al momento de renderizar la vista.
- **Cuerpo del HTML**: Aquí tenemos una estructura básica con un `<h1>` que dice "Hola desde Inicio". Esto es lo que verá el usuario cuando visite esta página. El contenido que se muestra en la vista puede ser dinámico, dependiendo de lo que el controlador pase.

2. Archivo `head.php`: Cabecera común para las vistas

Este archivo contiene los elementos de la cabecera del documento HTML, como las etiquetas `<meta>`, las hojas de estilo y los scripts.

Contenido de `head.php`:

```
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1">

<link rel="stylesheet" href="#PATH#css/main.css">

<script src="#PATH#js/jquery.js"></script>
<script src="#PATH#js/main.js"></script>
```

Análisis:

- **Meta etiquetas**: Define la codificación de caracteres y configura la vista del sitio para dispositivos móviles mediante la etiqueta `viewport`.
- **Hojas de estilo y scripts**: Incluye una hoja de estilo (`main.css`) y dos scripts, `jquery.js` y `main.js`, que son necesarios para el funcionamiento de la página. El marcador `#PATH#` es un placeholder que se sustituirá por la ruta correcta del sitio durante el proceso de renderizado.

3. Interacción Controlador-Vista

En el patrón **MVC**, el controlador es el responsable de pasar los datos a la vista para que se puedan renderizar correctamente. En este caso, dentro de `HomeController.php`, podemos ver cómo el controlador invoca la vista y le pasa los datos necesarios.

En la acción `actionInicio()` de `HomeController`:

```
public function actionInicio(){
    $head = SiteController::head();
    Response::render($this->viewDir(__NAMESPACE__), "inicio", [
        "title" => "Inicio MVC",
        "head" => $head,
    ]);
}
```

`Response::render()`: Este método se encarga de renderizar la vista. Se le pasa la ruta a la vista (`"inicio"`) y un array con los datos necesarios (`title` y `head`).

Datos que se pasan a la vista:

- `title`: Contiene el valor `"Inicio MVC"`, que será insertado en el título de la página.
- `head`: Contiene el HTML de la cabecera común (`head.php`), que es utilizado dentro de la sección `<head>` de la página.

4. Flujo completo de la interacción en MVC:

1. **Solicitud del usuario**: El usuario visita la URL que corresponde a la acción `actionInicio()` del controlador `HomeController`.
2. **Controlador**:
   - El controlador obtiene los datos necesarios para la vista (en este caso, el título y el contenido de la cabecera).
   - Llama a la función `Response::render()` para cargar la vista `inicio.php`, pasando los datos.
3. **Vista**:
   - La vista recibe los datos desde el controlador y los inserta en las posiciones correspondientes (`title` y `head`).
   - La vista genera el HTML final que será enviado al navegador del usuario.

Resumen:

- **Vista**: Es la capa de presentación que muestra la información al usuario. En tu aplicación, las vistas están organizadas en la carpeta `views` y contienen archivos PHP que generan HTML dinámico.
- **Controlador**: El controlador obtiene los datos desde el modelo y se los pasa a la vista, como en el caso del título y la cabecera en `inicio.php`.
- **Separación de responsabilidades**: El controlador se encarga de la lógica, el modelo de los datos, y la vista de cómo se muestran esos datos al usuario.

Este ciclo es esencial en el patrón MVC para mantener una separación clara entre la lógica de negocio (modelo), la interacción del usuario (vista) y la gestión de las solicitudes (controlador).

EL ABC: La lógica de programación en la aplicación MVC

Aclaraciones de cómo podemos usar el ejemplo dado en clases.

1. Rutas y Enrutamiento:

Tu aplicación sigue un esquema de URL en formato:

```
/controlador/metodo/parametro_1/parametro_2/parametro_N
```

Si no se especifica un **método** en la URL, se utiliza por defecto el método `actionIndex` del controlador correspondiente.

Los métodos que pueden ejecutarse desde la barra de direcciones siguen la convención de nombrado `actionNombreMetodo`.

El **controlador por defecto** es `HomeController`, y este se carga si no se pasa ningún controlador en la URL.

2. Convenciones de Controladores y Métodos:

- **Controladores**:
  - Todos los controladores siguen la convención `NombreController`. Esto facilita la carga automática de clases a través del autoloader y el sistema de enrutamiento en `App.php`.
  - No existen excepciones para esta convención, por lo que cualquier nuevo controlador debe seguir este formato.
- **Métodos**:
  - Los métodos que pueden ser accesibles desde la URL llevan el prefijo `action`. Si un método no tiene este prefijo, no puede ser accedido directamente desde la URL.
  - Solo los métodos públicos con el prefijo `action` son accesibles desde la URL.
  - Métodos sin `action`, aunque sean públicos, no son accesibles desde la URL y son usados dentro de la lógica interna de los controladores.

3. Manejo de errores (404 y otros):

- **`action404()`**: Este método existe tanto en el controlador base (`Controller.php`) como en el `HomeController.php`. Sin embargo, el que se usa en la práctica es el de `HomeController`.
- **Comportamiento cuando no se encuentra un controlador o método**:
  - Si el enrutador no encuentra el controlador o método solicitado en la URL, redirige automáticamente al método `action404()` del `HomeController`.
  - Este método se encarga de mostrar una página de error o vista específica, informando al usuario que la página solicitada no fue encontrada.

Esto asegura que cuando se produzcan errores como rutas no válidas, la aplicación no se rompa y el usuario vea una página de error personalizada.

5. Flujo completo de ejecución en tu aplicación MVC:

Vamos a resumir cómo interactúan todos estos componentes desde el momento en que el usuario realiza una solicitud hasta que el sistema carga un controlador o maneja un error.

Paso a paso:

1. **Solicitud del usuario**:
   - El usuario hace una solicitud en el navegador mediante una URL del formato `/controlador/metodo/parametro`.
2. **Punto de entrada (`index.php`)**:
   - El archivo `index.php` es el punto de entrada de la aplicación. Aquí se inicializa el entorno y se instancia la clase `App`:

```php
<?php
session_start();
// Habilitar el manejo de errores durante el desarrollo
error_reporting(E_ALL);
ini_set('display_errors', 1);

// Definir rutas principales
chdir(dirname(__DIR__));
define("CORE_PATH", "app/core/");
define("APP_PATH", "app/");
define("ROOT_PATH", "public/");

// Incluir el autoloader
require CORE_PATH . 'Autoloader.php';

// Iniciar la aplicación
new App();
```

3. Enrutamiento (`App.php`):

- La clase `App` se encarga de interpretar la URL utilizando el método `parseUrl()`.
- Determina el **controlador** a partir del primer segmento de la URL y el **método** a partir del segundo segmento.
- Si no hay un método en la URL, se utiliza el método por defecto `actionIndex`.

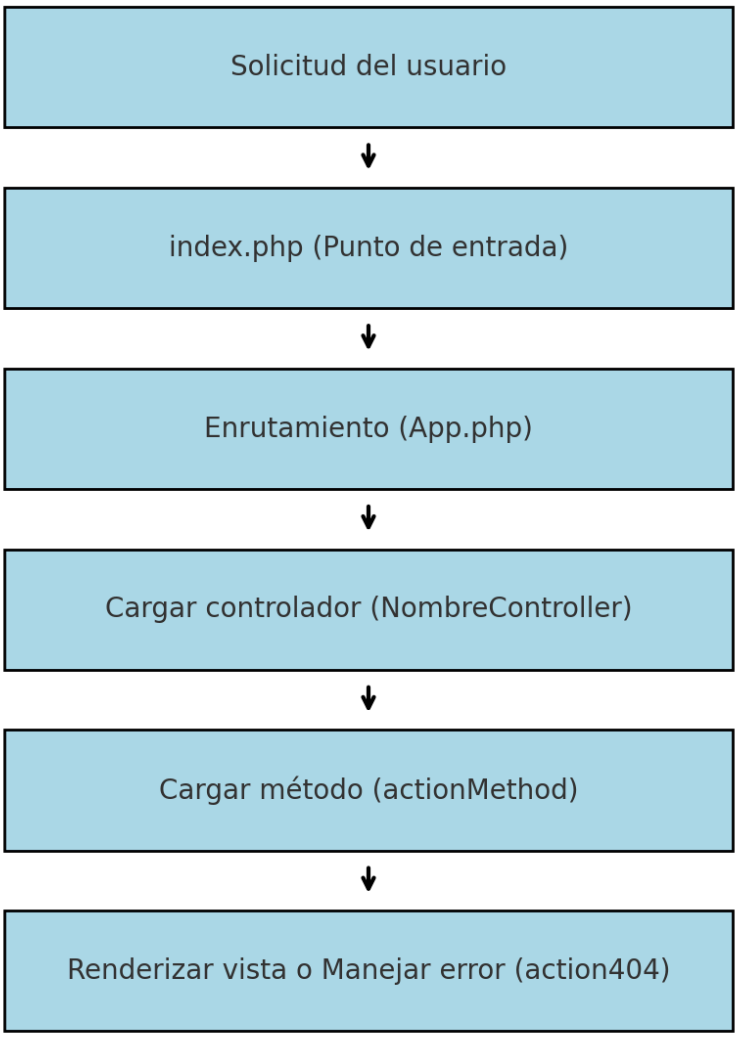
4. Carga del controlador y método:

- El autoloader carga automáticamente el archivo del controlador correspondiente siguiendo la convención `NombreController`.
- Si se encuentra el controlador, se instancia y se ejecuta el método con prefijo `action`.

5. Manejo de errores:

- Si el controlador o el método no se encuentran, `App.php` redirige a `HomeController::action404()`, que muestra una página de error 404 personalizada.

Flujo de ejecución en una aplicación MVC



1. **Solicitud del usuario**: El proceso comienza cuando el usuario realiza una solicitud desde la barra de direcciones.
2. **`index.php` (Punto de entrada)**: El archivo `index.php` es el punto de entrada de la aplicación, donde se inicializa el entorno y se carga la clase `App`.
3. **Enrutamiento (`App.php`)**: `App.php` se encarga de interpretar la URL, determinar el controlador y el método correspondientes.
4. **Cargar controlador**: El controlador específico (siguiendo la convención `NombreController`) se carga mediante el autoloader.
5. **Cargar método**: Si se encuentra el método con prefijo `action`, este se ejecuta. Si no hay un método, se llama a `actionIndex`.
6. **Renderizar vista o manejar error**: Si todo está correcto, se renderiza la vista correspondiente. Si no se encuentra el controlador o método, se llama a `action404` para manejar el error.

Análisis general del archivo App.php

Este archivo es clave en tu aplicación, ya que maneja el **enrutamiento** y la **invocación de controladores y métodos**. A continuación, te muestro una versión comentada del archivo, seguida de una explicación más detallada de cada sección.

Código de App.php comentado:

```
<?php
/**
 * Clase principal para manejar rutas y controladores
 */
class App
{
    // Propiedades para el controlador, método y parámetros
    protected $controller = "app\\controllers\\"."HomeController";  // Controlador por defecto
    protected $method = "actionIndex";  // Método por defecto
    protected $params = [];  // Parámetros que pueden ser pasados por la URL

    public function __construct()
    {
        // Extraer la URL y convertirla en un array
        $url = $this->parseUrl();

        // Obtener el nombre del controlador desde la URL (si existe)
        $controllerName = isset($url[0]) ? ucfirst(strtolower($url[0])) . "Controller" :
$this->controller;
        $controllerPath = APP_PATH . "controllers/" . $controllerName . ".php";

        // Verificar si el controlador existe
        if ($this->controllerExists($controllerPath)) {
            // Si existe, lo definimos como el controlador a utilizar
            $this->controller = APP_PATH . "controllers/" . $controllerName;
            unset($url[0]);  // Quitamos el controlador de la URL
        } else {
            // Si no existe el controlador, manejamos el error
            $this->handleControllerNotFound($url);
        }

        // Obtener la clase del controlador, utilizando el namespace adecuado
        $controller = $this->getControllerClass($this->controller);
        $this->controller = new $controller;

        // Manejar el método (acción) desde la URL
        $methodName = $this->getMethodFromUrl($url);
        $this->method = method_exists($this->controller, $methodName) ? $methodName : 'action404';  //
Si no existe, redirigir a 404

        // Parámetros restantes en la URL (si los hay)
        $this->params = $url ? array_values($url) : [];

        // Llamar al método del controlador con los parámetros
        call_user_func_array([$this->controller, $this->method], $this->params);
    }

    // Parsear la URL y devolverla como un array
    public function parseUrl()
    {
        return isset($_GET['url']) ? explode("/", filter_var(rtrim($_GET["url"], "/"),
FILTER_SANITIZE_URL)) : [];
    }
```

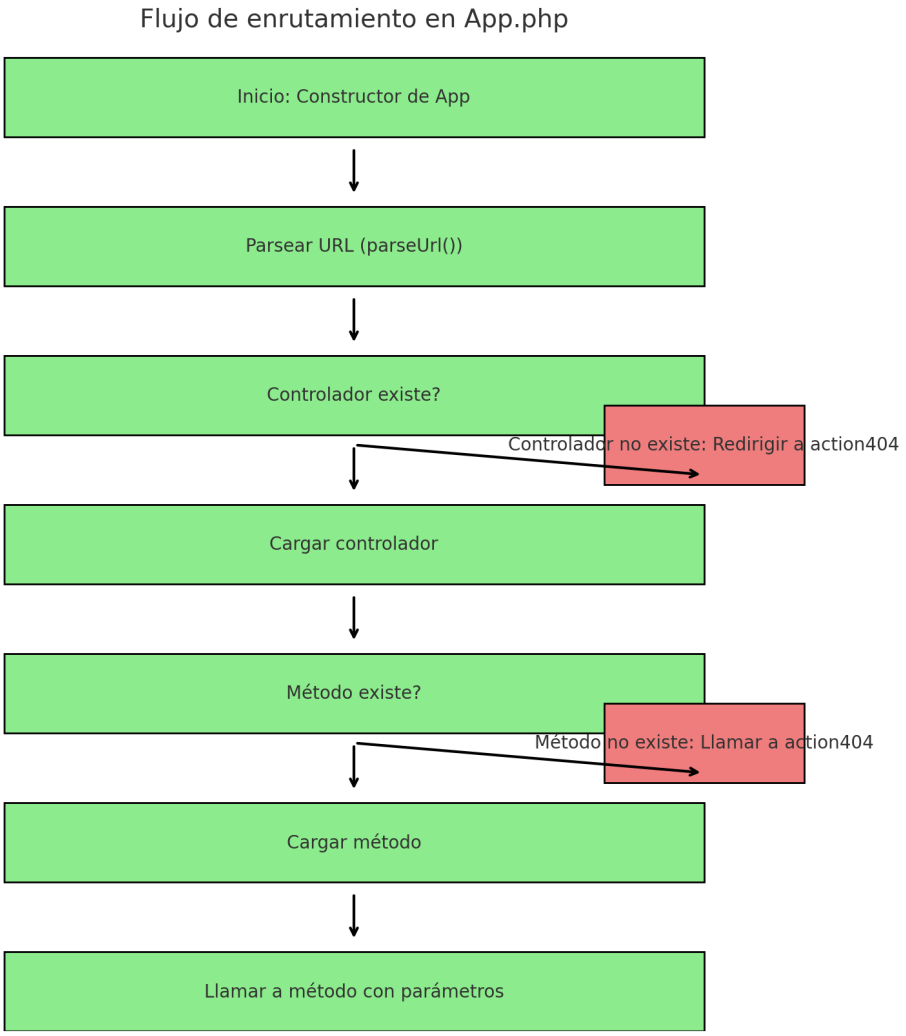
```
    // Verificar si el archivo del controlador existe
    protected function controllerExists($path)
    {
        return file_exists($path);
    }

    // Convertir el nombre del controlador a su espacio de nombres
    protected function getControllerClass($controllerPath)
    {
        return str_replace('/', '\\', $controllerPath);
    }

    // Manejar cuando el controlador no existe (redirigir a 404)
    protected function handleControllerNotFound(&$url)
    {
        if (isset($url[0])) {
            $this->method = 'action404';  // Redirigir a la página 404 si no se encuentra el
controlador
            unset($url[0]);
        }
    }

    // Obtener el nombre del método (acción) desde la URL
    protected function getMethodFromUrl(&$url)
    {
        return isset($url[1]) ? "action" . ucfirst(strtolower($url[1])) : $this->method;
    }
}
```

Desglose del código:

1. **Propiedades**:
   - `protected $controller`: Define el controlador por defecto, que es `HomeController`.
   - `protected $method`: Define el método por defecto, que es `actionIndex`.
   - `protected $params`: Almacena los parámetros adicionales de la URL.
2. **Método `__construct()`**:
   - Este método es el núcleo de la clase y se ejecuta automáticamente cuando se instancia la clase `App`.
   - **`$url = $this->parseUrl();`**: Extrae la URL ingresada por el usuario y la convierte en un array, separando cada segmento por "/".
   - **Controlador**:
     - Si se encuentra un controlador en la URL (primer segmento), se carga. Si no, se utiliza el controlador por defecto (`HomeController`).
   - **Método**:
     - El segundo segmento de la URL se interpreta como el método o "acción" del controlador.
     - Si no hay un método especificado, el método por defecto será `actionIndex`.
     - Si no se encuentra el método, se redirige a `action404`.
   - **Parámetros**:
     - Los segmentos restantes de la URL se consideran parámetros y se pasan al método.
     - La función `call_user_func_array` se usa para llamar al método del controlador con esos parámetros.
3. **Métodos auxiliares**:
   - **`parseUrl()`**: Convierte la URL en un array dividiéndola por "/".
   - **`controllerExists()`**: Verifica si el archivo del controlador existe en el sistema.
   - **`getControllerClass()`**: Convierte la ruta del controlador al formato de espacio de nombres adecuado.
   - **`handleControllerNotFound()`**: Redirige a `action404` si no se encuentra el controlador.
   - **`getMethodFromUrl()`**: Obtiene el método o acción desde la URL.

Explicación gráfica

Proceso que sigue la clase `App.php` para cargar un controlador y método específicos a partir de la URL.



Este es un **diagrama de flujo** que ilustra cómo la clase `App.php` maneja el enrutamiento y la ejecución de controladores y métodos:

1. **Inicio: Constructor de App**: El proceso comienza cuando se instancia la clase `App`, que inmediatamente llama a su constructor.
2. **Parsear URL**: El método `parseUrl()` convierte la URL ingresada por el usuario en un array.
3. **¿Controlador existe?**: Se verifica si el archivo del controlador solicitado existe. Si no existe, se redirige a `action404`.
4. **Cargar Controlador**: Si el controlador existe, se carga la clase del controlador correspondiente.
5. **¿Método existe?**: Se verifica si el método solicitado (con el prefijo `action`) existe dentro del controlador. Si no existe, se redirige a `action404`.
6. **Cargar Método**: Si el método existe, se carga.
7. **Llamar a método con parámetros**: Finalmente, el método es llamado con los parámetros proporcionados en la URL.

Si el controlador o el método no existen, el flujo redirige al método `action404` para manejar el error de manera apropiada.